Using the Record Mapper

Version 2026.1
2026-04-20

InterSystems Corporation One Congress Street Boston MA 02114 www.intersystems.com

Using the Record Mapper
PDF generated on 2026-04-20
InterSystems IRIS Version 2026.1

For Support questions about any InterSystems products, contact:

InterSystems Worldwide Response Center (WRC)
Tel: +1-617-621-0700
Tel: +44 (0) 844 854 2917
Email: support@InterSystems.com

Table of Contents

1 Record Mapper Overview 1
1.1 Basics 1
1.2 Complex Record Mapper 1
1.3 Record Map Batches 2
1.4 See Also 2

2 Using the Record Mapper 3
2.1 Creating and Editing a Record Map 3
2.1.1 Introduction 3
2.1.2 Getting Started 4
2.1.3 Common Control Characters 5
2.1.4 Editing the Record Map Properties 6
2.1.5 Editing the Record Map Fields and Composites 8
2.2 Using the CSV Record Wizard 11
2.3 Record Map Class Structure 12
2.4 Object Model for the RecordMap Structure 14
2.5 Using a Record Map in a Production 14
2.6 See Also 14

3 Using the Complex Record Mapper 15
3.1 Complex Record Mapper Overview 15
3.2 Creating and Editing a Complex Record Map 16
3.2.1 Getting Started 16
3.2.2 Editing the Complex Record Map Properties 16
3.2.3 Editing the Complex Record Map Records and Sequences 17
3.3 Complex Record Map Class Structure 18
3.4 Using a Complex Record Map in a Production 19
3.5 See Also 19

4 Record Map Batches 21
4.1 Creating Batches 21
4.2 See Also 22

1

Record Mapper Overview

The Record Mapper provides a quick, efficient way to map fixed-width or delimited text files to persistent production messages and back again. This page provides an overview.

1.1 Basics

The Record Mapper tool, available within the Management Portal, allows you to visually create a representation of a text file and create a valid object representation of that data which maps to a single persistent production message object. The process of generating both the target object structure and an input/output parser is automated, leaving only a few options for the persistent structure of the object projection. InterSystems IRIS® data platform generates the objects in such a way as to be a single persistent tree to provide complete cascading delete operations.

The Management Portal also provides a **CSV Wizard** to help you convert CSV (Comma Separated Value) files into a record map structure. This is particularly useful for files which contain column headers, as the wizard uses the header names to create the record map properties corresponding to the columns in the sample file.

The Record Mapper handles simple records which are either delimited or have fixed-width fields. A record map consists of a series of fields, which identify data in the record, and composites, which organize fields into a unit. In delimited records, the hierarchal level of composites specify the separator that is used between fields. Within delimited records, you can have simple fields that are repeating. You cannot have repeating composites. You can optionally ignore any fields in the incoming text file so that they do not waste space in the stored records.

The Record Mapper is not capable of dealing with mixtures of delimited and fixed-width data, nor is it capable of dynamically adjusting its parser or object structure based on the content of the incoming record other than handling repeating simple fields.

See Using the Record Mapper.

1.2 Complex Record Mapper

The Complex Record Mapper allows you to handle structured records containing different record types, including the ability to handle structures with repeating records and mixed delimited and fixed-field records. See Using the Complex Record Mapper.

1.3 Record Map Batches

An additional feature of the Record Mapper package allows you to batch heterogeneous records by implementing a class which inherits from EnsLib.RecordMap.Batch. This class handles parsing and writing out any headers and trailers associated with a specific batch. For simple headers and trailers, the Record Mapper user interface permits the creation of a batch of type EnsLib.RecordMap.SimpleBatch. You can extend either of these two batch implementations if you need to process more complex header and trailer data.

When a RecordMap batch operation is creating a batch from individual records, it stores the partially constructed batch in an intermediate file. You can specify the location of this file using the IntermediateFilePath property on the batch operation. On a mirrored system, you can store the intermediate file on a network drive that is accessible to both the main and failover system. Then if a failover occurs the failover system can continue to append records to the incomplete batch. Since the incomplete batch is stored in a file and not in an InterSystems IRIS database, it is not automatically copied to the mirror system. See Record Map Batches.

1.4 See Also

- Using the Record Mapper
- Using the Complex Record Mapper
- Record Map Batches

2

Using the Record Mapper

This page explains how to use the Record Mapper tool to work more easily with files containing delimited and fixed-width records, consisting of multiple fields.

Important: No field can exceed the long string limit.

2.1 Creating and Editing a Record Map

This section describes how to create and edit a record map. It contains the following sections:

- Introduction
- Getting Started
- Common Control Characters
- Editing the Record Map Properties
- Editing the Record Map Fields and Composites

2.1.1 Introduction

You can create a record map using the **Record Mapper** page of the Management Portal. If your file is a delimited file, you can also use the CSV Record Wizard to further automate the process. As you develop your record map, you can view how a sample file displays in the record map.

Important: Regenerating record maps (including CSVRecord maps) discards manual modifications to generated code. To help guide this principle, the generated parser class methods (GetObject, PutObject, GetRecord, PutRecord) are clearly marked with “DO NOT EDIT” comments.

The **Record Mapper** page includes a visual representation of the record map structure, and a simple interface which allows you to enter and manipulate the more detailed settings available for the record map components. It also allows you to reposition sibling elements (elements at the same level). One of the more important features of the user interface is that if you have a sample input file, a sample parse of the file will be attempted when the current record map is saved. You can address small issues in your record map directly from the Management Portal.

You can also create a record map directly using XML and the model classes.

2.1.2 Getting Started

To start the Record Mapper, select **Interoperability** > **Build** > **Record Mapper**. From here, you have the following commands:

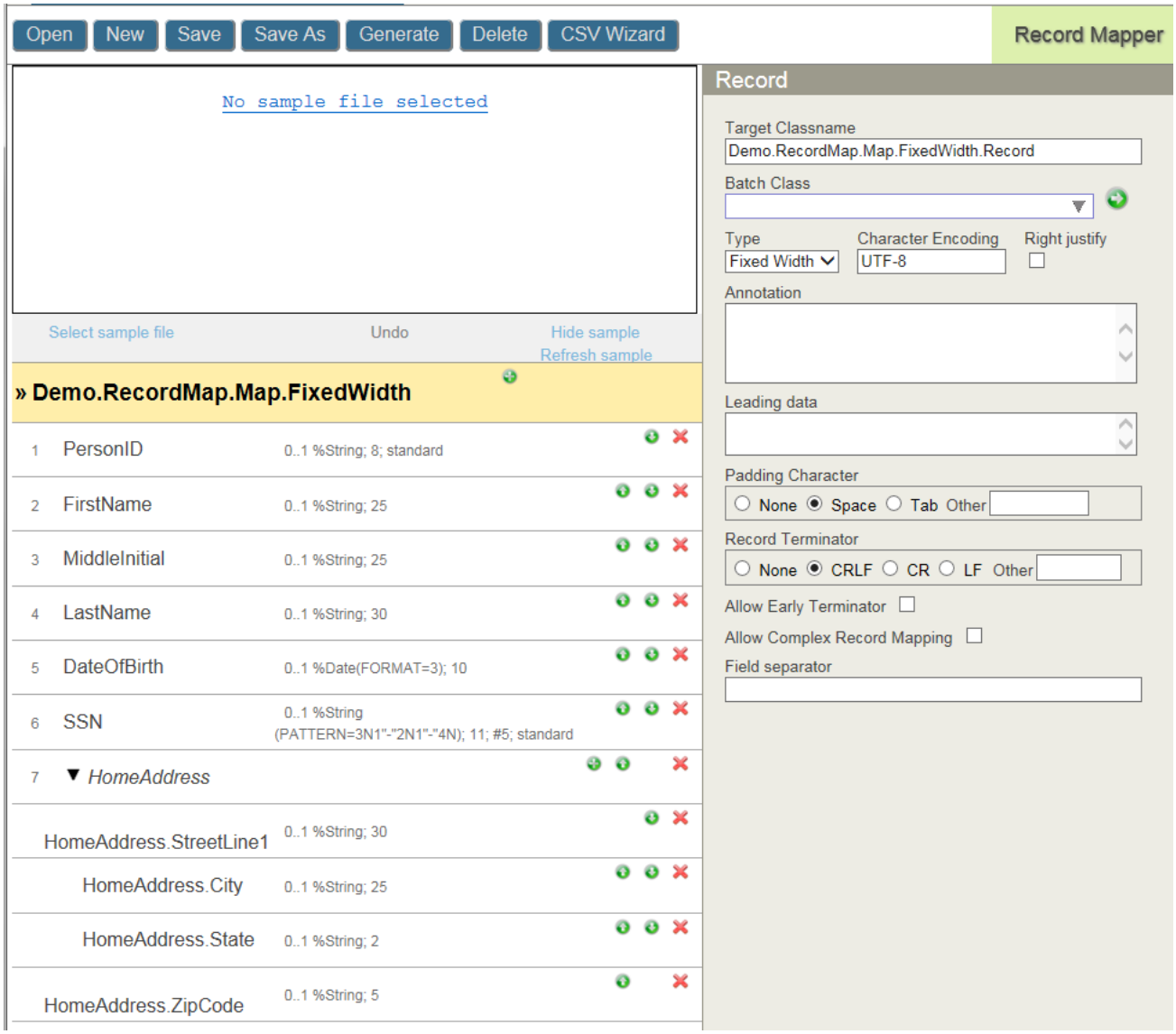
- **Open**—Displays the finder dialog box for you to choose an existing record map to open for editing.
- **New**—Initializes the page for you to enter a new record map structure.
- **Save**—Saves your record map structure as a class in the namespace in which you are working. Once saved, the object appears in the list of record maps.
- **Save As**—Saves your record map structure as a new class in the namespace in which you are working. Once saved, the object appears in the list of record maps.
- **Generate**—Generates the record map parser code and the related persistent message record class object.

  To generate an object manually use the **GenerateObject()** class method in EnsLib.RecordMap.Generator. It permits a number of options regarding the persistent structure of the generated objects, as noted in the comments for the method.
- **Delete**—Deletes the current record map. You can optionally delete the related persistent message record class and all stored instances of the classes.
- **CSV Wizard**—Opens the CSV Record Wizard to help automate the process of creating a record map from a sample file that contains comma separated values (CSV).

Important: The **Save** operation only writes the current record map to disk. In contrast, the **Generate** operation generates the parser code and the persistent object structure for the underlying objects.

When you select the name of the record map on the left side of the page, you see the record settings on the right, where you can edit the properties of the record map itself. Before you can save a record map, you must add at least one field to your record map. The following sections describe these processes:

Once you have created a new Record Map or opened an existing one, the Record Mapper displays a summary of the fields defined in the Record Map on the left panel and, on the right panel, allows you to set the properties of the Record Map or of the selected field. If you have specified a sample data file, it is displayed above the left panel. For example, the following shows the Record Mapper with the Record Map properties in the right panel:



To export, import, or delete a Record Map, click **Interoperability**, **List**, and **Record Maps** to display the Record Map Lists page.

2.1.3 Common Control Characters

Within a record map, you can use literal control characters as well as printable characters in several places. For example, you can specify a tab character, which is a common control character as well as a comma, which is a printable character, as a separator. You can also specify control characters as a padding character or as one of the record terminator characters. To specify a control character in one of these contexts, you must specify the hexadecimal escape sequence for the character. If you select the space or tab character as the padding character, or CRLF (carriage return followed by a line feed), CR, or LF as the record terminator character in the Record Mapper, the Management Portal automatically generates the hexadecimal representation. If you are specifying another control character as the padding character or in the record terminator or any control character as a separator, you must enter the hexadecimal representation in the corresponding form field. The following table lists the hexadecimal escape sequence for commonly used control characters:

Character	Hexadecimal representation
Tab	`\x09`
Line feed	`\x0A`
Carriage return	`\x0D`
Space	`\x20`

For additional characters, see https://en.wikipedia.org/wiki/C0_and_C1_control_codes or other resources.

Note: If you specify a record terminator in the RecordMap, the incoming message must match the record terminator exactly. For example, if you specify CRLF (\x0D\x0A), then the incoming message record must match that sequence.

2.1.4 Editing the Record Map Properties

Whether you are entering properties for a new record map, starting from the wizard-generated map, or editing an existing map, the process is the same. For the record itself, enter or update values in the following fields:

RecordMap Name

Name of the record map. You should qualify the record map name with the package name. If you do not provide a package name and specify an unqualified record map name, the record map class is saved in the User package by default.

Target Classname

Name of the class to represent the record. By default, the Record Mapper sets the target class name to a qualified name equal to the record map name followed by ".Record", but you can change the target class name. You should qualify the target class name with the package name. If you do not provide a package name and specify an unqualified target class name, the target class is saved in the User package by default.

Batch Class

Name of the batch class (if any) which should be associated with this record map.

Type

The type of record; options include the following:

- Delimited
- Fixed Width

Character Encoding

Character encoding for imported data records.

Right justify

Flag that specifies that padding characters should appear left of data in fields.

Annotation

Text that documents the purpose and use of the record map.

Leading data

Static characters which appear before any data of the actual record contents. If you are using the record map in a complex record map, you must identify the record with leading data.

Padding Character

Character used to pad the value. The padding character is removed by business services from the incoming message and used by business operations to pad the field value to fill fields in fixed-width record maps.

- None
- Space
- Tab
- Other

Record Terminator

Character or characters used to terminate the record.

- None
- CRLF
- CR
- LF
- Other

Allow Early Terminator (fixed-width record maps only)

Flag that specifies whether records can be terminated before the end. If allowed, record is treated as if it was padded with the padding character.

Allow Complex Batching

Flag that specifies whether record map can be used in a complex record map.

Field separator (fixed-width record maps only)

Optional single character used to separate fixed-width fields in records. If specified, input messages must contain this character between fields and business operations write this character between fields.

Field separator(s) (delimited record maps only)

A list of field separator characters. The first separator delimits the top-level fields in the record. The next separator delimits fields within a top-level composite field. Additional separators delimit fields within nested composite fields.

Repeat separator (delimited record maps only)

A single separator character that is used in all repeating fields.

Quoting: None (delimited record maps only)

Radio button that specifies there is no quote-style escaping.

Quote Escaping (delimited record maps only)

Radio button that enables quote-style escaping to allow a separator character to occur in a field value. Any input field can be quoted with the quote character. The field is considered all the characters between the start quote and end quote. Any separator character that appear within the quotes is treated as a literal character, not a separator. On output, any field that contains a separator in its value is quoted with the quote character.

Quote All (delimited record maps only)

Radio button that enables quote-style escaping to allow a separator character to occur in a field value. This has the same effect as **Quote Escaping** except that on output all fields are quoted whether they contain a separator character or not.

Quote character (delimited record maps with quote escaping only)

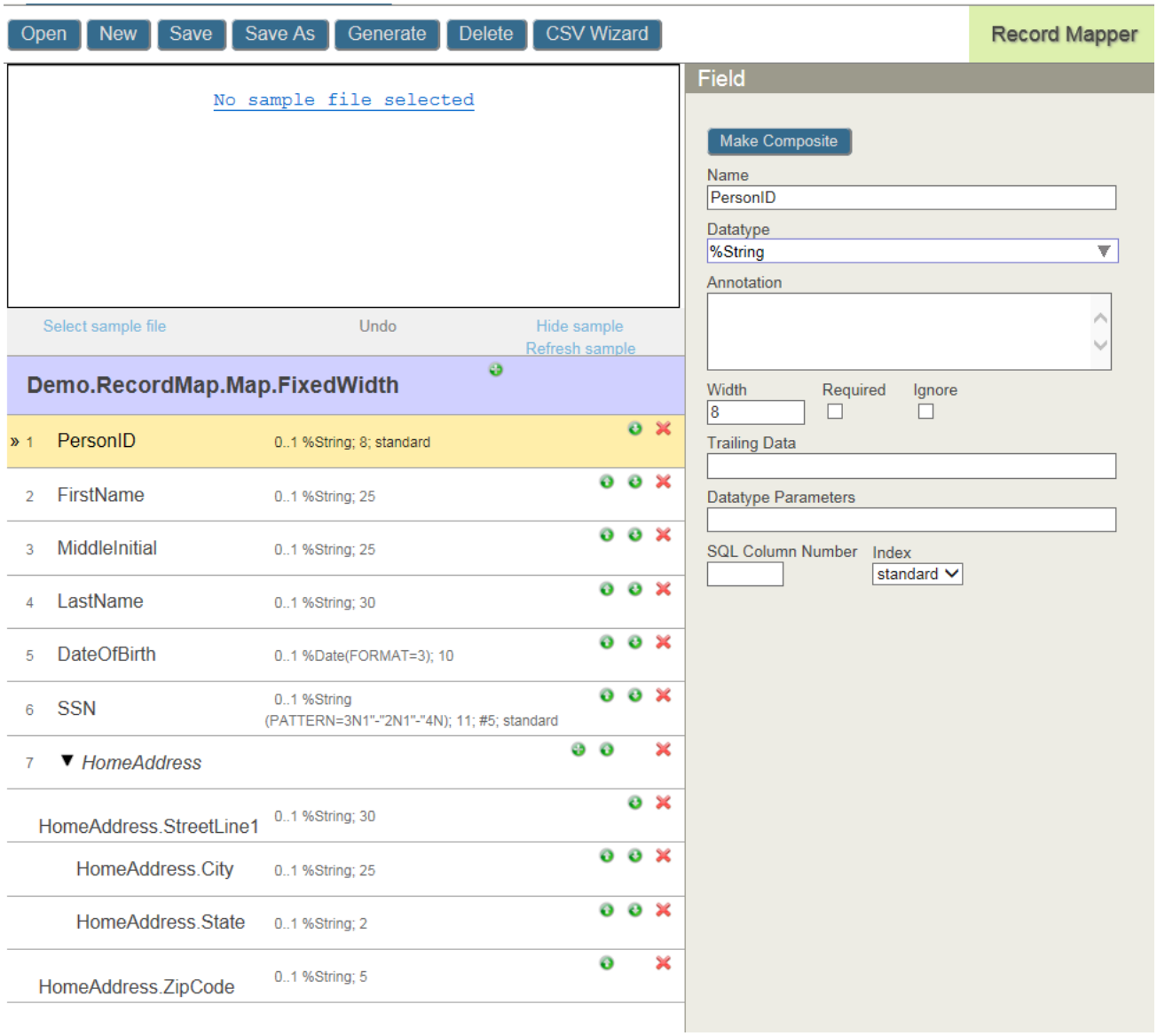
Character used to quote field contents. This field is displayed if you select the **Quote Escaping** or the **Quote All** radio button. If you are using a control character as a quote, you must enter it in hexadecimal; see Common Control Characters.

Allow Embedded Record Terminator

Determines what happens when the Record Mapper encounters the Record Terminator within a quoted field. If selected, the Record Mapper escapes the Record Terminator, treating it as part of the field data rather than considering it as the end of the record.

2.1.5 Editing the Record Map Fields and Composites

The Record Mapper left panel displays a summary of the fields defined in the Record Map. If you select a field, the right panel accesses the field properties. For example:



Record maps consist of a sequence of fields and composites. Each composite consists of a series of fields and composites. The **Make Composite** and **Make Field** buttons switch between a composite and a data field. For composite fields, you only specify the name and the flag indicating the field is required. Click the green plus sign icon on the record map to add a field or composite to the top level. Clicking on the plus sign of a composite allows you to add a field or composite to it.

While you are adding fields to your record map, you can open a sample file to see how its data maps to the record you are creating.

For delimited record maps, fields within composite fields have different separators. For example, in a record, the top-level field are delimited by commas, but within a composite the fields are delimited by semicolons. For fixed-width record maps, composite fields help organize the data conceptually, but do not impact the processing of the input message.

When you create a composite field in the Record Mapper, composite fields set the default name as a qualified name that matches the composite structure. The qualified field names determine the structure of fields within the generated record class. If you modify the field names to have different qualified names, the level of composite fields in the record map is independent from the structure of the fields in the generated record class.

For each data field you enter the following properties:

Name

Name of the field.

Datatype

Data type of the field. Select from the following list or enter a custom datatype:

- %Boolean
- %Date
- %Decimal
- %Double
- %Integer
- %Numeric
- %String
- %Time
- %Timestamp

Annotation

Documents the purpose and use of the field in the record map.

Width (fixed-width record maps only)

Width of the field.

Required

Flag that specifies that the field is required.

Repeating (delimited record maps only)

Flag that specifies that the field may contain repeated values using the record map's repeat separator character.

Ignore

Flag that specifies the field is ignored on input and not included in the stored record. Using the **Ignore** property saves storage space for the stored records. On output, InterSystems IRIS outputs an empty value for ignored fields—for fixed-width records, it fills the field with spaces, and for delimited records, it writes two consecutive separators for the empty field.

Trailing Data (fixed-width record maps only)

Characters that must follow this field. Control characters must be entered in hexadecimal; see Common Control Characters.

Datatype Parameters

Parameters (with their values) to apply to the data type. If you specify more than one parameter, separate them with a semicolon. For example:

```
DISPLAYLIST=,a,b,c;VALUELIST=,1,2,3;MAXLEN=''
```

For available parameters, see Common Property Parameters.

SQL Column Number

The SQL column number of the field. This value must either omitted or be between 2 and 4096 (inclusive) as per the values for the SqlColumnNumber property keyword. The column number is of particular use when importing data from CSV files or similar data dumps, as the SQL representation can be replicated easily.

Index

Enumerated value that controls whether the property should be indexed; select one of the following:

- (blank)—do not index
- 1
- bitmap
- idkey
- unique

The left panel of the Record Mapper is a table with a summary of the field definitions. The columns specify:

- Top-level field number.
- Field name.
- Summary of the properties of the field. The summary contains the following information, separated by ; (semicolon):
  - ignored—present if the **Ignore** check box is selected.
  - 0..1 or 1..1 followed by the datatype and datatype parameter—for optional or required fields, respectively.
  - Field width for fixed-width Record Maps.
  - #nnn—for SQL Column Number, if specified.
  - standard, bitmap, idkey, or unique—type of index, if specified.

  For example, an SSN field in a fixed-width Record Map could have a summary `0..1 %String(PATTERN=3N1"-"2N1"-"4N); 11; #5; standard`. This means it is an optional field, with a datatype and datatype parameters `%String(PATTERN=3N1"-"2N1"-"4N)`, has a field width of 11, has an SQL column number of 5, and has a standard index.
- Icons that allow you to move the field up or down or to delete the field. For composite fields, the plus icon allows you to add a new subfield.

2.2 Using the CSV Record Wizard

InterSystems IRIS provides a wizard to help automate the process of creating a record map from a sample file that contains comma separated values (CSV). You initiate the CSV Record Wizard either by choosing it on the InterSystems IRIS **Build** submenu or by clicking **CSV Wizard** from the ribbon bar on the **Record Mapper** page. The wizard handles only files with a single level of separator and does not handle leading data.

From the wizard, you enter values for the following fields:

Sample file

Either enter the complete path with filename of your sample or click **Select file** to navigate and choose your sample file.

RecordMap name

Enter the name of the record map to generate from your sample file.

Separator

Separator character used in the sample file. You must enter control characters in hexadecimal; see Common Control Characters.

Record Terminator

Specify how the sample file terminates a record. Choose one of the following:

- CRLF—each record ends with a carriage return, followed by a line feed.
- CR—each record ends with only a carriage return.
- LF—each record ends with only a line feed.
- Other—each record ends with control characters. Enter control character values in hexadecimal; see Common Control Characters.

Character Encoding

Select the type of character encoding used in the sample file.

Sample has header row

Select this check box if the sample file you provide contains a header row.

In this case, InterSystems IRIS removes any punctuation and white space from values in the header row, and then uses the resulting values as property names in the record map. (If you do not select this option, InterSystems IRIS specifies the property names as `Property1`, `Property2`, and so on.)

Keep SQL Column order

Select this check box to keep the SQL column order in the generated object.

Quote-style escaping in use

Select this check box and the quote character if the sample file uses quote-style escaping of the separator.

When you are finished filling out the wizard form, click **Create RecordMap** to generate a new record map from your sample file and return to the **Record Mapper** page. You can now refine your record map to add detail to the generated properties. See Editing the Record Map Properties for details.

2.3 Record Map Class Structure

There are two classes that describe a record map:

- RecordMap that describes the external structure of the record and implements the record parser and record writer.
- Generated record class that defines the structure of the object containing the data. This object allows you to reference the data in data transformations and in routing rule conditions.

A record map business service reads and parses the incoming data and creates a message, which is an instance of the generated record class. A business process can read, modify or generate an instance of the generated record class. Finally, a record map business operation uses the data in the instance to write the outgoing data using the RecordMap as a formatting

template. Both the RecordMap class and the generated record class have hierarchical structures that describe the data, but the generated object structure does not have to be identical to the RecordMap structure.

When you create a new record map and then save it in the Management Portal, this action defines a class for that extends the RecordMap class. In order to define the generated record class, you must click **Generate** in the Management Portal, which calls the **GenerateObject()** method in the EnsLib.RecordMap.Generator class. Just compiling the RecordMap class definition does not create the code for the generated record class. You must use the Management Portal or call the **Generator.GenerateObject()** method from the Terminal or from code.

The RecordMap consists of a sequence of fields and composites :

- A field defines a data field with the specified type. The field type can specify parameters such as, VALUELIST, MAXVAL, MAXLEN, and FORMAT. In fixed-width records, the Record Mapper uses the field width to set the default value for the MAXVAL or MAXLEN parameters.
- A composite consists of a sequence of fields and composites. Composites can be nested within a RecordMap.

By default, the Record Mapper in the Management Portal uses the composite level to set the qualified names of the fields. In delimited records, the nesting level of composites elements determines the separator used between fields as follows:

1. Fields in RecordMap that are not contained in a composite are delimited by the first separator.
2. Fields that occur in a composite that is in the RecordMap are delimited by the second separator.
3. Fields that occur in a composite that is itself within a composite are delimited by the third separator.
4. Each additional level of composite nesting increments the separator used to delimit the fields.

Composites in fixed-width records provide documentation about the structure of the data but do not impact how InterSystems IRIS treats the message.

Each RecordMap object has a corresponding record object structure. When you generate the RecordMap, the Record Mapper defines and compiles a record object that defines the object representation of the record map. By default, the Record Mapper in the Management Portal names the record “Record” qualified by the name of the RecordMap, but you can explicitly set of the name of the record object in the **Target Classname** field. By default, the Record Mapper names fields within composites by qualifying the name with the composites that contain it. If you use the default qualified names, the structure of the record object class properties will be consistent with the structure of the RecordMap fields and composites, but if you assign other names to the fields, the structure of the record object class properties will not match the structure of the RecordMap fields and composites.

The record object class extends the EnsLib.RecordMap.Base, %Persistent, %XML.Adaptor, and Ens.Request classes. If the *RECORDMAPGENERATED* parameter of the existing class is 0, then the target class is not modified by the record map framework—all changes are then the responsibility of the production developer. The properties in the generated record class are dependent on the names of the fields in the record map.

The properties of the record object class correspond to the fields of the record map and have the following names and types:

- Names of fields with simple unqualified names that appear anywhere in the RecordMap or in composites within it. These properties have a type determined by the type of the field.
- Top-level names of fields with qualified names that appear anywhere in the RecordMap or in composites within it. These properties have an object type with a class defined by the fields that share the same top-level qualified name. These classes extend the %SerialObject and %XML.Adaptor classes. These classes are defined within the scope of the generated record class name. These classes, in turn, have properties corresponding to the next level of name qualification.

Consider an example, where you are defining a delimited record map, where the data contains three levels of separators, such as where the top-level separator field delimits the information about a person, the next level delimits the information

about identification number, name, and phone number; and the final level delimits the elements within the address and name. For example, the message could start with:

```
French Literature,TA,199-88-7777;Jones|Robert|Alfred;
```

To define a RecordMap to handle these separators, you would need a composite at the level of person and one at the level of name. Thus the default field name for the FamilyName field could be Person.Name.FamilyName. This default name creates a deep level of class names in the record object class, such as the class NewRecordMap.Record.Person.Name that contains properties such as NewRecordMap.Record.Person.Name.FamilyName. You can avoid this deep level by prefacing the field names with the $ (dollar sign) character. If you do this, the classes and properties are all defined directly in the record scope. Using the same example, the class NewRecordMap.Record.Name would contain properties such as NewRecordMap.Record.FamilyName.

Note: The names used to qualify the field names are used to define properties with an object type. Consequently, you cannot use a name both to qualify a field name and to be the last part of a field name, which would define a property with the same name with the same data type.

2.4 Object Model for the RecordMap Structure

You can achieve the class structure behavior either by directly creating XML or by using the EnsLib.RecordMap.Model.* classes to create an object projection of the RecordMap. In general, the favored approach is to use Management Portal, but you may prefer to use the object model to create the RecordMap structure. The structure of these classes follows the RecordMap class structure; use the class reference for further information at this level.

2.5 Using a Record Map in a Production

When you choose to generate an object class on the Record Mapper page, you create a class that you can use in a business service of a production.

2.6 See Also

- Using the Complex Record Mapper
- Record Map Batches

3

Using the Complex Record Mapper

With the Record Mapper, you may need to handle messages whose format consists of multiple heterogeneous records. Typically, these complex records have a header record followed by a pattern of records and terminated by a trailer record. These records can either be fixed-fields records or delimited records and can be optional and repeat. Typically, the records have leading data that identifies the kind of record.

Important: No field can exceed the long string limit.

3.1 Complex Record Mapper Overview

Complex record maps can describe structured records that can contain:

1. An optional header record.
2. Sequence of elements where each element can be a record defined by a RecordMap or a sequence. A sequence can contain a sequence of records and other sequences.
3. An optional trailer record.

The records within a sequence can either be delimited records or fixed-width records. Although it is possible to mix delimited and fixed-width records within a complex record map, typically all of the records are delimited or all are fixed-width.

The following delimited sample data can be described by a complex record map. The data consists of a header identifying a semester in a college and information about students and classes that each student takes.

```
SEM|194;2012;Fall;20
STU|12345;Adams;John;Michael;2;john.michael.adams@example.com;617-999-9999
CLS|18.034;1;Differential Equations;4
CLS|21W.759;1;Writing Science Fiction;4
STU|12346;Adams;Jane;Michelle;3;jane.michelle.adams@example.com;
CLS|21L.285;1;Modern Fiction;3
CLS|7.03;1;Genetics;4
STU|12347;Jones;Robert;Alfred;1;bobby.jones@example.com;
CLS|18.02;1;Calculus;4
```

The complex record map that describes this data consists of:

1. Header record identified by the leading data “SEM|”.
2. A sequence of students, where each student consists of:
    a. Student record identified by the leading data “STU|”.

b. A repeating class record identified by the leading data “CLS|”.

The sequence of students defines the repeating structure of the complex record but does not correspond to a record in the data.

A complex record map defines both a file structure and an object structure. The complex record map file service parses a file using the file structure defined by the complex record map and then stores the data in an object defined by the object structure. The complex record map file operation performs the reverse—it takes the data in the object and writes it out to a file using the file structure defined by the complex record map.

3.2 Creating and Editing a Complex Record Map

You can create a new complex record map or edit an existing one by using the Complex Record Mapper in the Management Portal or by importing an XML definition and editing one in an IDE.

The complex map has a top-level sequence in which you can define the fields. If your complex record consists of a single sequence of records, where the sequence does not repeat, you can enter the record maps of the records directly in the complex record map. But, if the entire sequence can repeat between the header and trailer, then you should enter a repeating sequence record as the only element of the top-level sequence.

3.2.1 Getting Started

To access the Complex Record Mapper from the Management Portal, click **Interoperability**, **Build**, and **Complex Record Maps**. From here, you have the following commands:

- **Open**—Opens an existing complex record map.
- **New**—Creates a new complex record map.
- **Save**—Saves your complex record map structure as a class in the namespace in which you are working in the package specified in the **Complex RecordMap Name**.
- **Generate**—Generates the complex record map parser code and the related persistent message complex record class object.

  To generate an object manually use the **Generate()** class method in EnsLib.RecordMap.ComplexGenerator.
- **Delete**—Deletes the current complex record map. You can optionally delete the related persistent message complex record class and all stored instances of the classes.

Important: The **Save** operation only writes the current complex record map to disk. In contrast, the **Generate** operation generates the parser code and the persistent object structure for the underlying objects. Generating complex record maps will discard manual modification to generated parser code. To help guide this principle, generated classes (including header, footer, and batch) are clearly marked with “DO NOT EDIT” comments.

3.2.2 Editing the Complex Record Map Properties

Whether you are entering properties for a new complex record map, starting from the wizard-generated map, or editing an existing map, the process is the same. For the complex record itself, enter or update values in the following fields:

Complex RecordMap Name

Name of the complex record map. You should qualify the complex record map name with the package name. If you do not provide a package name and specify an unqualified complex record map name, the complex record map class is saved in the User package by default.

Target Classname

Name of the class to represent the complex record. By default, the Complex Record Mapper sets the target class name to a qualified name equal to the complex record map name followed by ".Batch", but you can change the target class name. You should qualify the target class name with the package name. If you do not provide a package name and specify an unqualified target class name, the target class is saved in the User package by default.

Character Encoding

Character encoding for imported data records. The specified for the complex record map should be the same as the encoding for all record maps included in the complex record map. If they are not the same, the character encoding for the complex record map overrides the character encoding for the record maps.

Annotation

Text that documents the purpose and use of the complex record map.

If you create a new complex record map, the Complex Record Mapper creates a definition that consists of the following elements.

- Complex map name and type—enter the name and class for the complex map.
- Header—if your complex record has a header, enter the name and the class of the record map that describes the header.
- Trailer—if your complex record has a trailer, enter the name and the class of the record map that describes the trailer.

3.2.3 Editing the Complex Record Map Records and Sequences

A complex record map consists of the following:

1. An optional header record.
2. Sequence of elements where each element can be:
   - Record defined by a RecordMap. It can have the following properties in the complex record map:
     - Required. A value 0 means the record is optional and a value 1 means it is required.
     - Repeating with a minimum and maximum number of occurrences.
   - A nested sequence of elements.
3. An optional trailer record.

Each record is defined by a record map. A sequence is defined in the complex record map definition. It describes the structure of the data in the message but does not itself correspond to any fields in the data.

The header and trailer records are each defined by a record map. Although it is optional to include a header or trailer record in the complex record map definition, if the definition contains a header record, then the data must contain a header record, and if the definition contains a trailer record, then the data must contain a trailer record. Header and trailer records cannot repeat.

Every sequence must contain at least one record or sequence.

When you are editing a record, you can click the **Make Sequence** button to replace the record with a sequence. When you are editing a sequence, you can click the **Make Record** button to replace the sequence with a record.

You can specify the following properties for a record:

- Record name.
- RecordMap that defines the record format. The RecordMap specifies the **Leading data** that identifies the record, whether the record has fixed columns or is delimited, the separators, and the record terminator. For details on defining a RecordMap, see Using the Record Mapper.
- Whether the record is required.
- Whether the record can be repeated. If the record can be repeated, you can also specify:
  - Minimum number of repetitions
  - Maximum number of repetitions
- Annotation that documents the purpose and use of the record in the complex record map.

You can specify the following properties for a sequence:

- Sequence Name
- Whether the sequence is required.
- Whether the sequence can be repeated. If the sequence can be repeated, you can also specify:
  - Minimum number of repetitions
  - Maximum number of repetitions
- Annotation that documents the purpose and use of the sequence in the complex record map.

3.3 Complex Record Map Class Structure

There are two classes that describe a complex record map in a similar manner to the two classes that describe a record map. The two classes that describe a complex record map are:

- Complex record map that describes the external structure of the complex record and implements the complex record parser and writer.
- Generated complex record class that defines the structure of the object containing the data. This object allows you to reference the data in data transformations and in routing rule conditions.

A complex record map business service reads and parses the incoming data and creates a message, which is an instance of the generated record class. A business process can read, modify or generate an instance of the generated complex record class. Finally, a complex record map business operation uses the data in the instance to write the outgoing data using the complex record map as a formatting template. Both the complex record map class and the generated complex record class have hierarchical structures that describe the data. The complex record map class and the generated complex record class have parallel structures. This is different from the RecordMap class, where the generated record class can have a different hierarchal structure.

When you create a new complex record map and then save it in the Management Portal, this action defines a class for that extends the EnsLib.RecordMap.ComplexMap and Ens.Request classes. In order to define the generated record class, you must click **Generate** in the Management Portal, which calls the **Generate()** method in the EnsLib.ComplexGenerator class. Just compiling the ComplexMap class definition does not create the code for the generated record class. You must use the

Management Portal or call the **ComplexGenerator.Generate()** method from the Terminal or from code. The generated class extends the RecordMap.ComplexBatch and Ens.Request classes.

The ComplexMap class defines the complex record structure in an XData definition that defines the ComplexBatch with records specified by RecordReference elements and sequences defined by RecordSequence elements. If the *RECORDMAPGENERATED* parameter of the existing class is 0, then the target class is not modified by the complex record map framework—all changes are then the responsibility of the production developer.

The ComplexBatch class has properties that correspond to the following top-level elements in the complex map definitions:

- Header record, if specified. This property has its type set to the generated record class for the specified record map.
- Record that has its type set to the generated record class for the specified record map or, if the record can be repeated, its type is set to an array of the generated record class.
- Sequence that has its type set to the class defined for the sequence or, if the sequence can be repeated, its type is set to an array of this class.
- Trailer record, if specified. This property has its type set to the generated record class for the specified record map.

A class is defined for each sequence. The sequence class extends the ComplexSequence and %XML.Adaptor classes. The sequence class is defined within the package and namespace defined for the ComplexBatch class. All sequence classes are defined in this level of the namespace even if they are contained within other sequences.

Each sequence has properties that correspond to the records and sequences that it contains.

3.4 Using a Complex Record Map in a Production

To create a production that uses complex records, you do the following:

1. Create the individual record maps for each part of the complex record, including the header and trailer. See Using the Record Mapper for a description of how to create the individual record maps. Note that if you intend to use a sample file, you should create a sample file that contains only the part of the complex record that you are defining in the individual record map. The sample file should not contain a complete complex record.
2. Use the Complex Record Mapper to define the structure of the complex record.
3. Create a production and add one or more of the built-in complex record services and operations.
4. If your production is simply passing a complex record from one application to another, you may be able to use a simple routing engine process. But, if your production is converting one complex record into a different complex record, you will create a Data Transformation in the routing engine. If both the input and output complex records have the same structure, you can create a simple data transformation that connects the source and target fields. For example, you could use a simple data transformation to convert a complex record containing delimited records to a complex record containing fixed column records. But if the input complex record does not have the same structure as the output complex record, you must add code either within the data transformation or in a Business Process Language (BPL) process.

3.5 See Also

- Using the Record Mapper
- Record Map Batches

4

Record Map Batches

The RecordMap feature imports a single record at a time, but if you are importing or exporting a large number of records, you can gain substantial efficiency improvement by using RecordMap Batch. The RecordMap Batch feature handles homogeneous records and processes all of the records in a batch at one time. The batch can optionally be preceded by a header record and followed by a trailer record.

4.1 Creating Batches

To create a RecordMap batch, you implement a class which inherits from %Persistent and EnsLib.RecordMap.Batch. The Batch class contains methods that handle parsing and writing out any headers and trailers associated with a specific batch. You must provide code that parses and writes your headers. For simple headers and trailers, you can use the EnsLib.RecordMap.SimpleBatch class, which inherits from the Batch class and provides code for handling simple headers and trailers. You can extend either of these two batch implementations if you need to process more complex header and trailer data.

Batch processing follows the approach used for other production message formats like X12. This is particularly relevant for the built-in business operations which handle RecordMap batch objects: these business operations accept either Batch objects or RecordMap objects which extend EnsLib.RecordMap.Base, or a request of type BatchRolloverRequest. When records in a particular batch are received, the Batch is opened and the batch header is written to a temporary file, followed by any objects within that batch received by the operation. If the request is synchronous, the classname, Id, and the count of previously written records for the batch will be returned in a EnsLib.RecordMap.BatchResponse. Receipt of the batch object (which may be the default batch) will trigger the batch trailer to be written to the temporary file, and this file will then be sent to the desired destination by the adapter for the business operation. If a Batch object is received by itself, then the entire Batch will be written out to a temporary file which will then be transferred to the desired location.

Important: If ArchiveIO is enabled during this process, a copy of the in bound stream will be saved in the temporary stream location for the namespace (the default location is <*install-dir*>/mgr/GLOBAL_DB_DIRECTORY/stream/) These streams will be purged automatically and should not be manually removed.

Batch operations also support a default batch option whereby records which do not already belong to a batch are added to a default batch. Output of this batch can be triggered by either sending the batch object to the operation, or sending a BatchRolloverRequest to the operation. The business operation can also be configured to use schedule- or count-based rollover for the default batch. These options are configured on the business operation, and can be used simultaneously.

The options for services primarily concern the way the Visual Trace displays messages within a batch.

The RecordMap Batch operation creates temporary files in the process of generating the final output file. You can control the location of these temporary files by specifying the **IntermediateFilePath** setting of the RecordMap Batch operation. If

the namespace's database is being mirrored, it is important that all mirror members have access to the temporary file in order to successfully failover during a RecordMap Batch operation. See the High Availability Guide for information on mirroring.

Important: RecordMap Batch messages use a one-to-many relationship to hold the records and this makes it quite easy to traverse all records and perform the desired transformation. However, the process can consume enough memory that you can receive <STORE> errors. You may need to increase the memory of the process, or split the input files, or implement customized transformations that use SQL instead of the one-to-many relationship.

4.2 See Also

- Using the Record Mapper
- Using the Complex Record Mapper